

Proyecto Alke Wallet

Módulo: Fundamentos de Bases de Datos Relacionales

Conny Benítez Ossa

Fecha: 25-08-2026

1. Diseño de la base de datos

Entidades: Usuario, Moneda y Transacción. Se agrega el campo `currency_id` (FK a Moneda) en Usuario para representar la moneda elegida por cada usuario, y `fecha_creacion` como campo adicional (tarea plus).

1.1 Creación de la base de datos

```
CREATE DATABASE AlkeWallet;
USE AlkeWallet;
```

1.2 Tabla: moneda

```
CREATE TABLE moneda (
  currency_id INT AUTO_INCREMENT PRIMARY KEY,
  currency_name VARCHAR(50) NOT NULL,
  currency_symbol VARCHAR(5) NOT NULL
);
```

1.3 Tabla: usuario

```
CREATE TABLE usuario (
  user_id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  correo_electronico VARCHAR(150) NOT NULL UNIQUE,
  contrasena VARCHAR(255) NOT NULL,
  saldo DECIMAL(12,2) NOT NULL DEFAULT 0.00,
  currency_id INT NOT NULL,
  CONSTRAINT fk_usuario_moneda FOREIGN KEY (currency_id) REFERENCES moneda(currency_id)
);
```

1.4 Tabla: transaccion

```
CREATE TABLE transaccion (
  transaction_id INT AUTO_INCREMENT PRIMARY KEY,
  sender_user_id INT NOT NULL,
  receiver_user_id INT NOT NULL,
  importe DECIMAL(12,2) NOT NULL,
  transaction_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_transaccion_sender FOREIGN KEY (sender_user_id) REFERENCES usuario(user_id),
);
```

```

CONSTRAINT                                fk_transaccion_receiver
FOREIGN KEY (receiver_user_id) REFERENCES usuario(user_id),
INDEX      idx_sender_receiver (sender_user_id, receiver_user_id)
);

```

1.5 Tarea Plus: agregar fecha de creación a usuario

```

ALTER TABLE usuario
ADD COLUMN fecha_creacion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP;

```

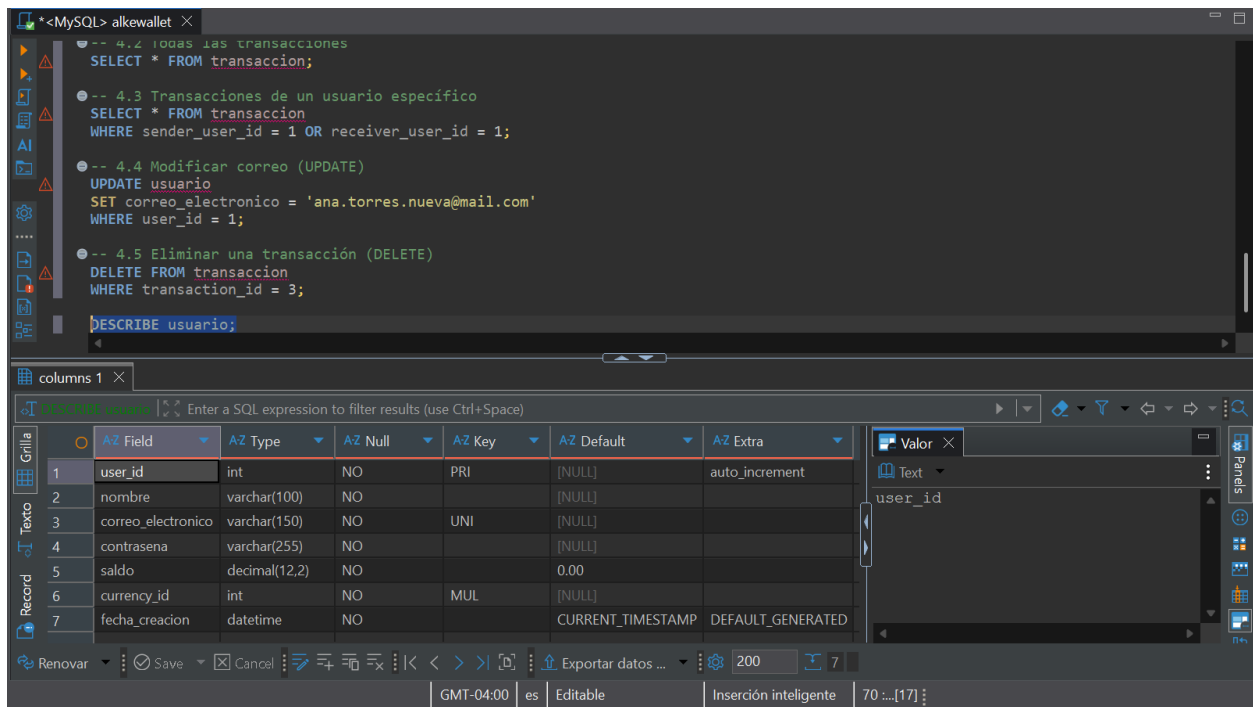


Figura 1: captura de SHOW CREATE TABLE usuario;

2. Inserción de datos de prueba (DML - INSERT)

2.1 Monedas

```

INSERT INTO moneda (currency_name, currency_symbol) VALUES
('Peso Argentino', '$'),
('Dólar Estadounidense', 'US$'),
('Euro', '€');

```

2.2 Usuarios

```

INSERT INTO usuario (nombre, correo_electronico, contrasena, saldo, currency_id) VALUES
('Ana Torres', 'ana.torres@mail.com', 'hashed_pwd_1', 1500.00, 1),
('Bruno Diaz', 'bruno.diaz@mail.com', 'hashed_pwd_2', 800.50, 2),
('Carla Ruiz', 'carla.ruiz@mail.com', 'hashed_pwd_3', 2300.75, 1);

```

2.3 Transacciones

```
INSERT INTO transaccion (sender_user_id, receiver_user_id, importe, transaction_date)
VALUES
(1, 2, 150.00, '2026-08-01 10:15:00'),
(2, 3, 75.50, '2026-08-05 14:30:00'),
(1, 3, 200.00, '2026-08-10 09:00:00');
```

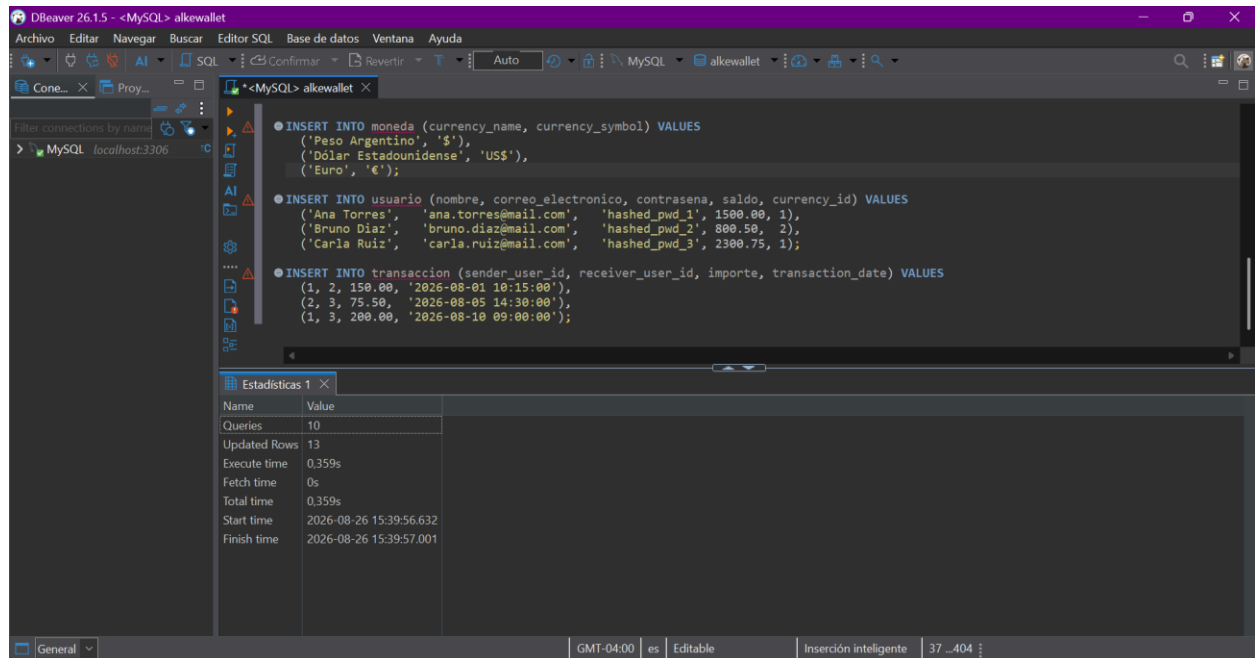


Figura 2: INSERT ejecutados con éxito (mensaje de filas afectadas)..

3. Manipulación de datos y transaccionalidad (ACID)

Ejemplo de transacción controlada para actualizar el saldo de dos usuarios al registrar una transferencia, con posibilidad de revertir ante un error.

```
START TRANSACTION;

UPDATE usuario SET saldo = saldo - 100.00 WHERE user_id = 1;
UPDATE usuario SET saldo = saldo + 100.00 WHERE user_id = 2;

INSERT INTO transaccion (sender_user_id, receiver_user_id, importe)
VALUES (1, 2, 100.00);

COMMIT;
-- En caso de detectar un error de integridad (por ejemplo, saldo
-- insuficiente o un user_id inexistente), se ejecuta ROLLBACK;
-- en lugar de COMMIT; para revertir todos los cambios de la transacción.
```

4. Consultas SQL solicitadas

4.1 Nombre de la moneda elegida por un usuario específico

```
SELECT      u.nombre,      m.currency_name,      m.currency_symbol
FROM        usuario u
INNER JOIN  moneda m      ON      u.currency_id = m.currency_id
WHERE u.user_id = 1;
```

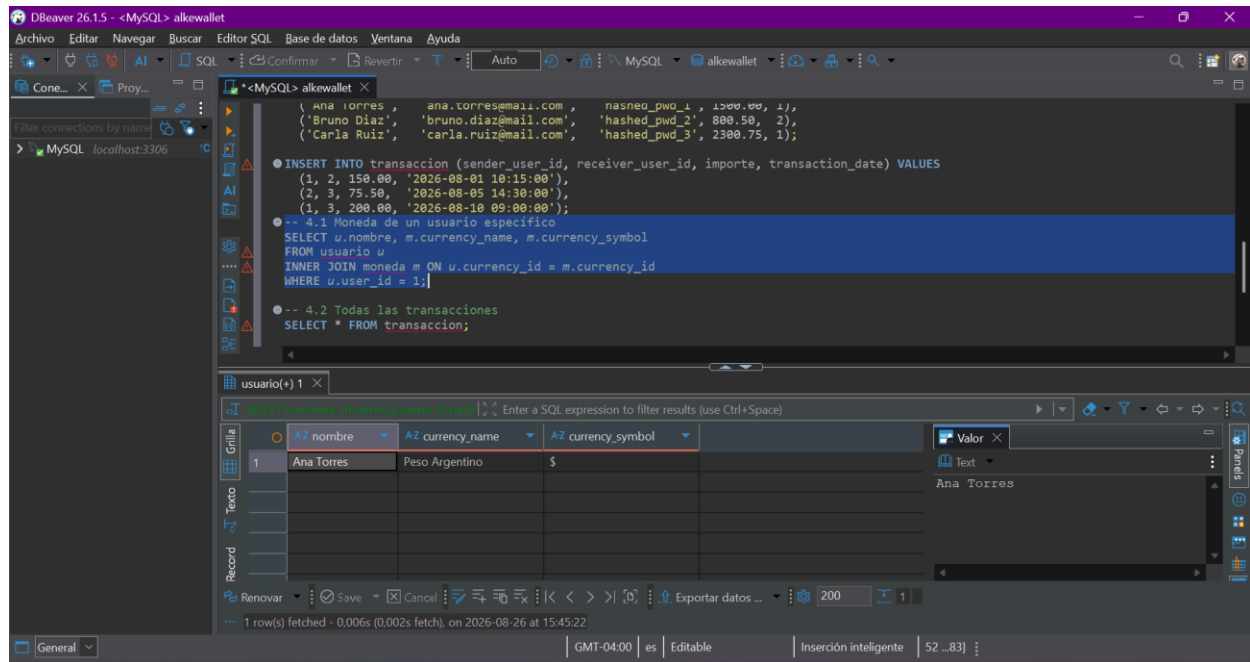


Figura 3: Ejemplo

4.2 Todas las transacciones registradas

```
SELECT * FROM transaccion;
```

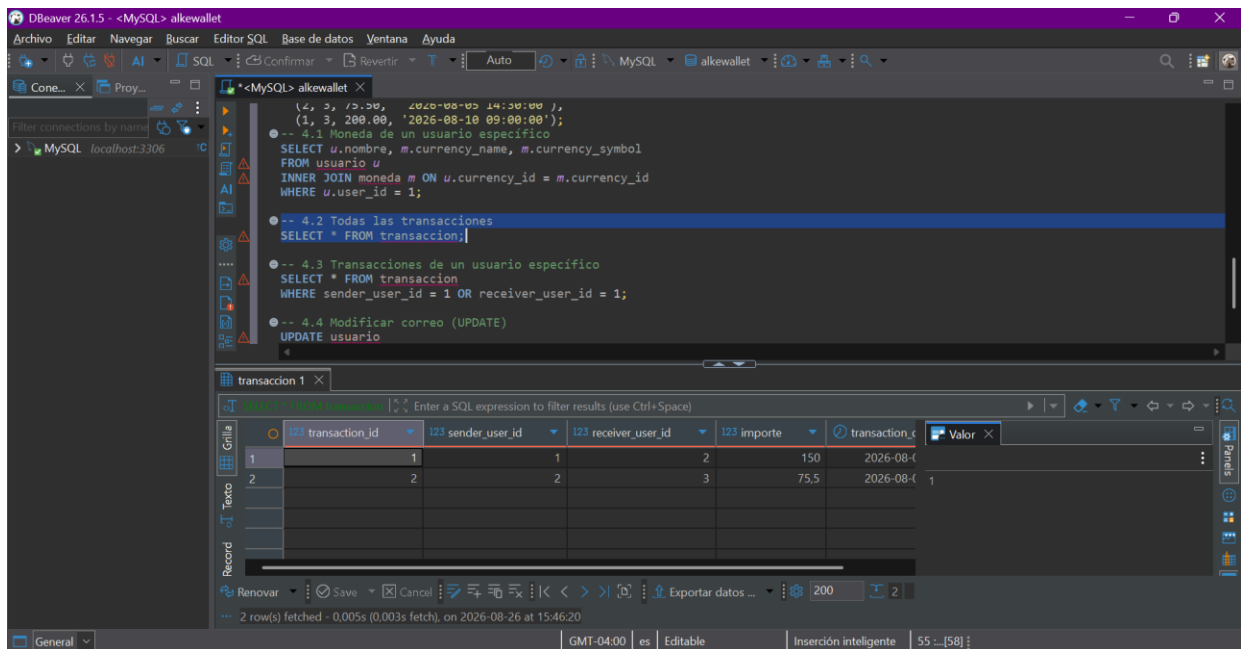


Figura 4: Ejemplo

4.3 Transacciones realizadas por un usuario específico

```
SELECT *
FROM transaction
WHERE sender_user_id = 1
OR receiver_user_id = 1;
```

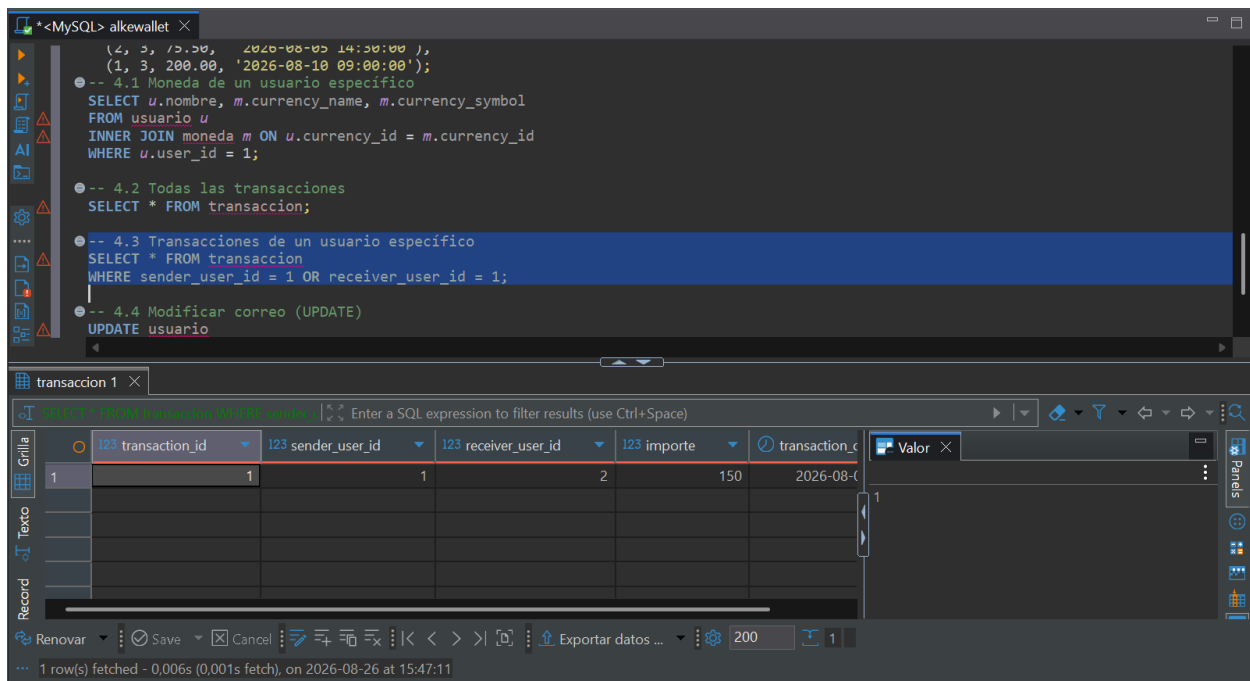
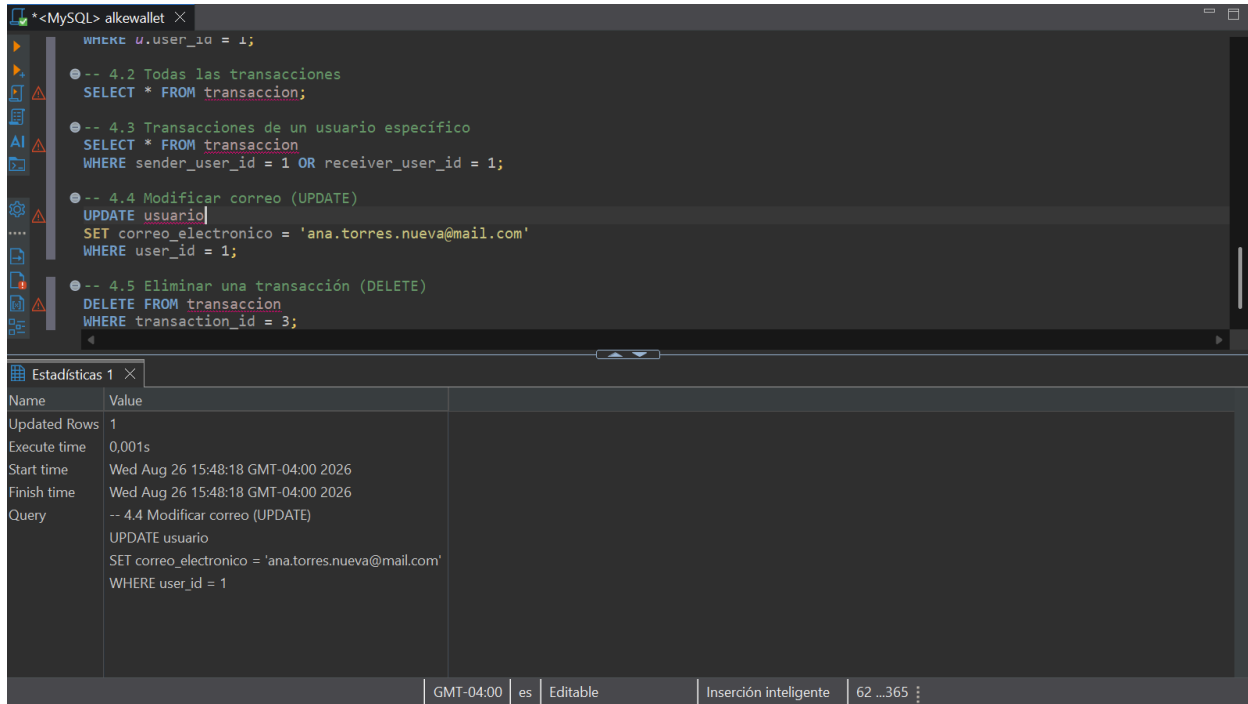


Figura 5: Ejemplo

4.4 Modificar el correo electrónico de un usuario específico (DML - UPDATE)

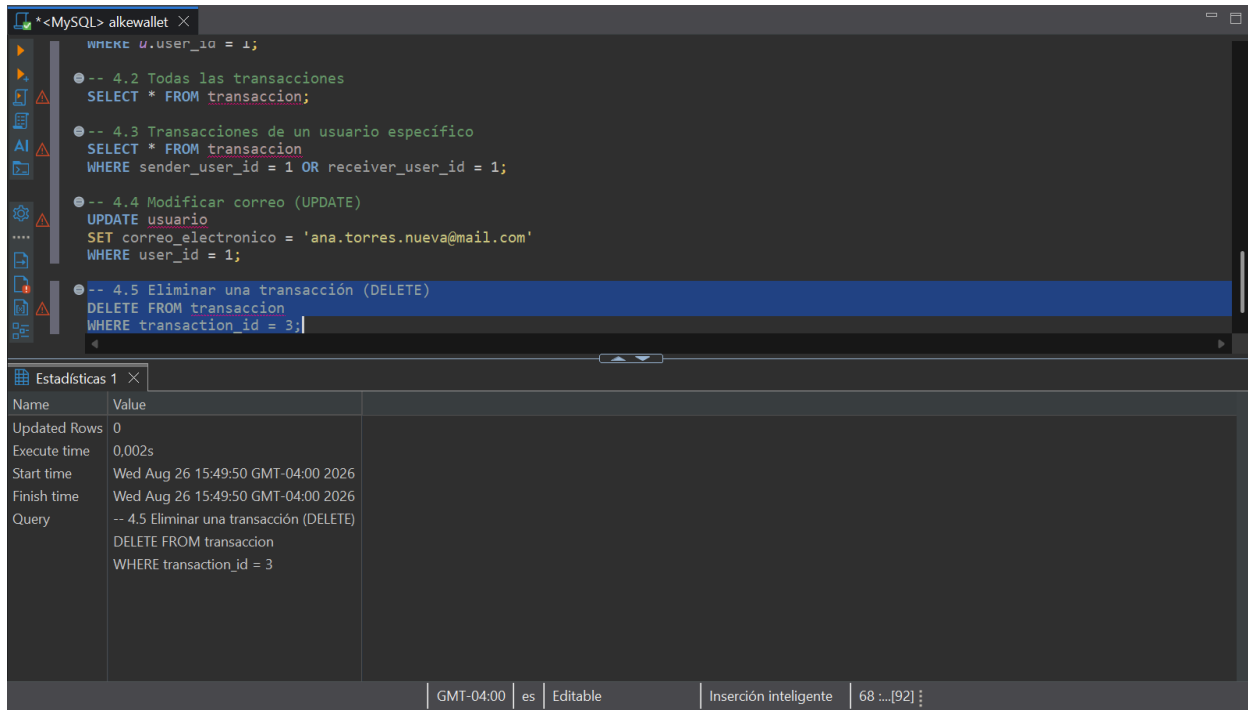
```
UPDATE
SET      correo_electronico = 'ana.torres.nueva@mail.com'
WHERE user_id = 1;
```



 **Figura 6:** Ejemplo

4.5 Eliminar los datos de una transacción (DML - DELETE)

```
DELETE                                FROM                                transaccion
WHERE transaction_id = 3;
```



The screenshot shows a MySQL IDE window titled '<MySQL> alkewallet'. The SQL editor contains several queries, with the last one highlighted: `-- 4.5 Eliminar una transacción (DELETE)`, `DELETE FROM transaccion`, `WHERE transaction_id = 3;`. Below the editor, the 'Estadísticas 1' (Statistics 1) window is open, displaying the following information:

Name	Value
Updated Rows	0
Execute time	0.002s
Start time	Wed Aug 26 15:49:50 GMT-04:00 2026
Finish time	Wed Aug 26 15:49:50 GMT-04:00 2026
Query	-- 4.5 Eliminar una transacción (DELETE) DELETE FROM transaccion WHERE transaction_id = 3

The status bar at the bottom indicates 'GMT-04:00', 'es', 'Editable', 'Inserción inteligente', and '68 ...[92]'.

 **Figura 7:** Ejemplo

5. Aspectos validados

- Diseño de la base de datos: modelo relacional con 3 entidades y relaciones definidas mediante claves foráneas.
- Integridad de los datos: claves primarias AUTO_INCREMENT, restricciones NOT NULL y UNIQUE (correo electrónico).
- Integridad referencial: claves foráneas de transaccion hacia usuario y de usuario hacia moneda.
- Uso de SQL: sentencias DDL (CREATE DATABASE, CREATE TABLE, ALTER TABLE) y DML (INSERT, SELECT, UPDATE, DELETE).
- Principios ACID: ejemplo de transacción con START TRANSACTION / COMMIT / ROLLBACK para garantizar atomicidad y consistencia.